

Emu Launcher

Kendi Arayüzünü Tasarla

Tauri + React + Tailwind projesinde UI'yi baştan yazma rehberi

1. Bu Rehber Ne İşine Yarar?

Bu belge, Emu Launcher'ın arayüzünü (UI) sıfırdan kendin tasarlamak ve yazmak isteyen sana yol gösterir. Amaç: mevcut çalışan mantığı (emülatör kaydetme, başlatma, klasör tarama, konsol logoları) bozmadan, yalnızca görsel katmanı istediğin gibi baştan kurabilmen.

Projenin altın kuralı şudur: mantık ile görsel ayrıdır. Emülatörleri diske kaydeden, .exe başlatan, klasör tarayan kod arka planda (Rust) ve ince bir köprü katmanında (backend.ts) durur. Sen bu köprüye birkaç fonksiyon çağırısıyla bağlanırsın, üstüne istediğin arayüzü çizersin. Bu rehber tam olarak o sınırın nerede olduğunu ve nasıl bağlanacağını anlatır.

Kısa özet

Dokunacağın yer: src/App.tsx, src/components/ ve src/index.css. Dokunmayıp sadece çağıracağın yer: src/backend.ts, src/systems.ts, src/artwork/.

2. Büyük Resim: Mimari

Uygulama üç katmandan oluşur. Arayüzü değiştirmek, yalnızca en üstteki katmanı değiştirmek demektir:

Katman	Teknoloji	Sorumluluk
Görsel (UI)	React + Tailwind	Ekranda ne görünür, nasıl görünür. Senin alanın.
Köprü	backend.ts / systems.ts / artwork	JS ile Rust arasındaki ince arayüz. Hazır fonksiyonlar.
Çekirdek	Rust (Tauri)	Dosya okuma/yazma, .exe başlatma, klasör tarama.

Veri akışı tek yönlü ve basittir:

```

Kullanıcı tıklar -> React bileşeni -> backend.ts fonksiyonu
                        |
                        v
                    Tauri (Rust) komutu
                        |
                        v
                Disk / .exe / klasör tarama
  
```

Yani yeni bir arayüz yazarken hiçbir zaman Rust'a dokunmana gerek yoktur. Sadece backend.ts içindeki hazır fonksiyonları çağırırsın.

3. Dokunma: Değişmez Backend Sözleşmesi

Hangi arayüzü yazarsan yaz, aşağıdaki fonksiyonlar senin "API"ndir. Bunları içe aktarıp (import) çağırırsın; imzaları değişmez. Hepsi src/backend.ts içinden gelir:

Fonksiyon	İmza	Ne yapar
loadEmulators	() => Promise<Emulator[]>	Kayıtlı emülatörleri diskten yükler.
saveEmulators	(list) => Promise<void>	Tüm listeyi diske yazar (her değişiklikte çağır).
launchEmulator	(exePath) => Promise<void>	Verilen .exe'yi başlatır.

Fonksiyon	İmza	Ne yapar
<code>pickExecutable</code>	<code>() => Promise<string null></code>	Native .exe dosya seçici açar.
<code>pickDirectory</code>	<code>() => Promise<string null></code>	Native klasör seçici açar.
<code>scanDirectory</code>	<code>(dir) => Promise<ExeEntry[]></code>	Klasördeki tüm .exe'leri bulur.

Veri modeli

Tek merkezi tip Emulator'dır. Her arayüz bu şekli kullanır (`src/types.ts`):

```
export interface Emulator {
  id: string;          // crypto.randomUUID() ile üret
  name: string;        // görünen ad, ör. "PCSX2"
  exePath: string;     // tam .exe yolu
  systemId?: string;  // sistem id'si, ör. "ps2" (opsiyonel)
}
```

Kalıcılık kuralı

State'i güncellediğinde hemen `saveEmulators(...)` çağır. Uygulama açılışında ise bir kez `loadEmulators()`. Diskteki JSON şema-bağımsızdır; ileride yeni alan eklersen backend'e dokunman gerekmez.

4. Frontend Katmanları: Neyi Değiştir, Neyi Bırak

Frontend dosyalarını rolüne göre ikiye ayır. "Görsel" olanları serbestçe yeniden yaz; "mantık" olanları olduğu gibi bırakıp sadece çağır:

Dosya	Rol	Yeniden tasarlar
<code>src/backend.ts</code>	Backend köprüsü (mantık)	DOKUNMA — sadece çağır
<code>src/types.ts</code>	Veri modeli	Genelde dokunma
<code>src/systems.ts</code>	Sistem katalogu + algılama	Dokunma (renkleri isteğe göre)
<code>src/artwork/</code>	Görsel çözümleme katmanı	DOKUNMA — çağır
<code>src/components/*.tsx</code>	Görsel bileşenler	SERBEST — burayı yeniden yaz
<code>src/App.tsx</code>	Düzen + durum yönetimi	Yeniden yaz (akışı koru)
<code>src/index.css</code>	Tema + global stil	Serbestçe değiştir

Neden `systems.ts` ve `artwork`'e dokunmuyoruz?

Bunlar "PCSX2 hangi sisteme ait, o sistemin logosu hangi dosya" bilgisini tutan veri/mantık katmanıdır. Arayüzün sadece sonucu ister: bir logo URL'si ve bir sistem adı. O yüzden onları çağırırsın, içlerini değiştirmesin.

5. Geliştirme Döngüsü: Nasıl Çalışır ve Görürüm?

İki farklı geliştirme yolun var. Görsel iterasyon için ilki çok daha hızlıdır:

A) Tarayıcıda hızlı iterasyon

```
npm run dev      # http://localhost:1420 (tarayıcıda açılır)
```

Bu modda uygulama tarayıcıda çalışır ve backend'i taklit eder: emülatörler localStorage'a kaydedilir, pickDirectory() sahte bir "C:\Emulators" yolu döndürür, scanDirectory() örnek bir liste verir. Gerçek .exe başlatmak çalışmaz ama tüm görsel tasarımı anında, hot-reload ile görebilirsin. UI çizerken çoğu zaman buradasın.

B) Gerçek masaüstü uygulaması

```
# terminal cargo'yu görmüyorsa önce (bir kez):
$env:Path = [Environment]::GetEnvironmentVariable('Path','User') + ';' + \
            [Environment]::GetEnvironmentVariable('Path','Machine')

npm run tauri dev      # gerçek pencere, gerçek dosya seçici ve başlatma
```

Rust tarafını değiştirmedeğin sürece görsel çalışman için (A) yeterlidir. Kod kaydettiğinde her iki modda da ekran otomatik yenilenir.

6. Tema ve Tailwind Temelleri

Renkler tek yerde, src/index.css içindeki @theme bloğunda tanımlıdır. Buradaki her değişken, Tailwind'de bir sınıf olur (ör. --color-accent → bg-accent, text-accent, border-accent):

```
@theme {
  --color-surface: #0b0d12;      /* en arka zemin */
  --color-surface-raised: #12151d; /* kartlar, paneller */
  --color-surface-hover: #1a1e29; /* hover zemini */
  --color-stroke: #232838;      /* kenarlıklar */
  --color-accent: #6d8dff;      /* vurgu (butonlar, linkler) */
}
```

Temayı komple değiştirmek istersen (ör. açık tema ya da farklı vurgu rengi) genelde tek yapman gereken bu beş değeri düzenlemektir; tüm bileşenler bu sınıfları kullandığı için otomatik uyum sağlar.

Tailwind v4 notu

Bu proje Tailwind v4 kullanır: ayrı bir tailwind.config.js yoktur, tema doğrudan CSS'te @theme ile tanımlanır. Temada olmayan bir renk için köşeli parantez sözdizimini kullanabilirsin: bg-[#1e2536], text-[13px].

7. Yaklaşım Seçimi: İki Yol

Yeni arayüze başlamanın iki yolu var. Hangisini seçersen seç, backend sözleşmesi aynı kalır:

Yol 1 — Mevcut bileşenleri değiştir (önerilen)

- `src/components/EmulatorCard.tsx` içindeki kart görünümünü değiştir.
- `src/App.tsx` içindeki grid/düzeni yeniden düzenle.
- Modallar (`EmulatorModal`, `ScanReviewModal`) hazır çalışıyor; istersen yalnızca stillerini güncelle.
- Avantaj: state yönetimi ve tüm akış zaten kurulu; sadece görüntüyü değiştirirsin.

Yol 2 — Sıfırdan yeni bir App yaz

- `App.tsx`'i boşaltıp kendi bileşen ağacını kur.
- Sadece `backend.ts`, `systems.ts` ve `artwork`'ten import et.
- Aşağıdaki minimal örnek, çalışan bir launcher için gereken her şeyi gösterir.

8. Adım Adım: Minimal Çalışan Arayüz

Aşağıdaki tek dosya, gerçek bir launcher için yeterli olan çekirdeği gösterir: emülatörleri yükler, logolarıyla bir grid çizer ve tıklanınca başlatır. Kendi tasarımını bunun üstüne kurabilirsin.

```
import { useEffect, useState } from "react";
import { loadEmulators, launchEmulator } from "../backend";
import { getSystem, systemLogo } from "../systems";
import type { Emulator } from "../types";

export default function App() {
  const [items, setItems] = useState<Emulator[]>([]);

  // Açılışta bir kez diskten yükle
  useEffect(() => { loadEmulators().then(setItems); }, []);

  return (
    <div className="grid grid-cols-4 gap-4 p-6">
      {items.map((e) => (
        <button
          key={e.id}
          onClick={() => launchEmulator(e.exePath)}
          className="rounded-xl border border-stroke bg-surface-raised p-4"
        >
          <img src={systemLogo(e.systemId)}
            alt={getSystem(e.systemId)?.name ?? e.name} />
          <span className="text-white">{e.name}</span>
        </button>
      ))}
    </div>
  );
}
```

Dikkat: burada yalnızca üç yerden import var — `backend`, `systems`, `types`. Arayüzün tamamı bu kadar basit bir yüzeye bağlanır.

Ekleme / silme / düzenleme

Listeyi değiştirmek için sadece React state'ini güncelle ve `saveEmulators(yeniListe)` çağır. Örn. ekleme:

```
const y = [...items, {id: crypto.randomUUID(), name, exePath, systemId}]; setItems(y);
saveEmulators(y);
```

9. Bağlanacağıın Uçlar (Hızlı Başvuru)

Sistemler ve logolar — src/systems.ts

Ne	Kullanım	Döner
SYSTEMS	SYSTEMS	Tüm sistemlerin listesi { id, name, color }
getSystem	getSystem(id)	Sistem nesnesi ya da undefined
systemLogo	systemLogo(id)	Gömülü logo URL'si ya da undefined
detectSystem	detectSystem(name, exe)	Ad/exe'den tahmini sistem id'si
identifyEmulator	identifyEmulator(dosyaAdi)	Bilinen emülatör tanımı

Görseller — src/artwork

Logoyu doğrudan systemLogo(id) ile de alabilirsin. Daha genel yol, ileride oyun kapaklarını da kapsayacak olan çözümleme katmanıdır:

```
import { resolveArtworkSync } from "../artwork";

const logo = resolveArtworkSync({ kind: "system-icon", systemId: e.systemId });
// logo: string | null -> <img src={logo} /> ya da yoksa yedek göster
```

İleriye dönük

Oyun kısayolları eklendiğinde aynı resolveArtwork arayüzü SteamGridDB'den oyun kapağı/logosu çekecek şekilde genişleyecek (src/artwork/steamGridDb.ts iskeleti hazır). Arayüz kodun değişmeden çalışmaya devam eder.

10. Launcher Arayüzü İçin Tasarım İlkeleri

Playnite, Steam Big Picture veya Heroic gibi launcher'ların ortak, işe yarayan kalıpları:

- Grid, listeye tercih edilir. Kapak/logo odaklı kartlar tarama hızını artırır. Responsive için auto-fill/minmax kullan.
- Sabit en-boy oranı. Kartların görsel alanı aynı orana (ör. 4:3 ya da kapak için 2:3) otursun ki grid hizalı görünsün.
- Hover'da eylem göster. Başlat/düzenle/sil butonlarını normalde gizle, üzerine gelince yumuşak bir geçişle göster; ekranı sakın tut.
- Boş durumu tasarla. Hiç emülatör yokken kullanıcıya ne yapacağını açıkça söyle (ekle / klasör tara).
- Klavye ve odak. Kartlar button olsun; ok tuşları + Enter ile gezinme launcher hissini güçlendirir.
- Geri bildirim. Başlatma/hata durumlarını küçük bir toast ya da durum çubuğuyla bildir.
- Tutarlılık. Köşe yuvarlaklığı, boşluk ölçeği ve renkler için temadaki değerlere sadık kal.

Performans

Ağır UI kütüphanelerinden kaçın (proje bilinçli olarak Tauri + hafif React kullanıyor). Yüzlerce kart olursa animasyonları sadeleştir; çok büyük listelerde (200+) sanallaştırma (windowing) düşün. Native his için ani, kısa geçişler (150-200ms) yeterlidir.

11. Sık Yapılan Hatalar ve İpuçları

- `systemId` opsiyoneldir. `systemLogo(id)` null dönebilir; her zaman bir yedek (monogram, ikon) göster.
- `saveEmulators`'ı unutma. Sadece state'i güncellemek yetmez; kalıcılık için diske de yazmalısın.
- `launchEmulator` yalnızca masaüstünde çalışır. Tarayıcı modunda (`npm run dev`) hata fırlatır; bunu bir toast ile yakala.
- id üretimi. Yeni emülatör için `crypto.randomUUID()` kullan; dizinin `index`'ini id olarak kullanma.
- Windows yolları. `exePath`'te ters bölü (`\\`) vardır; string işlerken kaçış karakterlerine dikkat et.
- Tema dışı renk. Gerektiğinde `bg-[#. . .]` ile serbest renk verebilirsin ama tutarlılık için önce temayı düşün.

12. Kaynaklar

- Tauri 2 (komutlar, pencere, dağıtım): <https://tauri.app>
- React (bileşenler, hooks): <https://react.dev>
- Tailwind CSS v4 (tema, sınıflar): <https://tailwindcss.com>
- Konsol logoları (gömülü set): <https://github.com/HVR88/Monochrome-Gaming-Logos>
- SteamGridDB (ileride oyun kapakları için): <https://www.steamgriddb.com>

Bu rehber, projenin mevcut kod yapısına göre hazırlanmıştır: dosya yolları ve fonksiyon imzaları buradaki haliyle geçerlidir. Kodda büyük bir yeniden yapılandırma yaparsan bu belgeyi de güncellemeyi unutma.